

SKYHIGH AIR: VIVA & PRESENTATION DEFENSE GUIDE

કોલેજના સર / એક્સટર્નલ એક્ઝામિનર સામે પ્રોજેક્ટ સમજાવવાની માસ્ટર ગાઇડ

Team Members: 1. Sahil (Frontend) | 2. Darshan (Backend API & DB) | 3. Krish (Security & DevOps)

1. ઓપનિંગ સ્પીચ (પ્રેઝન્ટેશનની શરૂઆત કેવી રીતે કરવી)

કોણ બોલશે: **SAHIL**

"Respected Sir / Examiners, Good morning/afternoon.

Today, our team—myself **Sahil**, along with my team members **Darshan** and **Krish**—presents our Final Year Major Project titled '**SkyHigh Air: Enterprise Online Airline Reservation & Passenger Fleet Management System**'.

Sir, the primary aim of our project is to modernize commercial airline operations by decoupling the frontend presentation layer from the backend API. We have developed this project using **React 18 with Vite** on the frontend, **PHP 8.2 REST API** on the backend, and **MySQL Database** with enterprise-grade security."

2. ટીમ મેમ્બર્સનો વ્યક્તિગત સ્ક્રિપ્ટ (કોણ કયો ભાગ સમજાવશે)

1. સાહિલ (Sahil) — ફ્રન્ટએન્ડ અને UI/UX ડિઝાઇન

SPEAKING ROLE: SAHIL

"Sir, I was responsible for the **Frontend Architecture & UI/UX Engineering** of SkyHigh Air.

1. We chose **React 18 with Vite** because traditional airline websites are slow due to full-page reloads. With React's Single Page Application (SPA) architecture and Virtual DOM, our page navigation is instant (sub-50ms).

2. Instead of boring dropdown lists for seat selection, I engineered an **Interactive 3-3 Cabin Layout** modeled after an **Airbus A320neo**. Passengers can visually see Available

seats, Booked seats, and their Selected seat in real time.

3. Our UI features modern glassmorphism, responsive navigation for mobile and desktop, flight search filters with real-time price sliders, and engaging micro-interactions like confetti animations upon successful booking."

ગુજરાતીમાં યાદ રાખો: React 18 કેમ વાપર્યું (ફાસ્ટ છે અને રીલોડ થતું નથી), 3-3 એરબસ સીટ મેપ કેવી રીતે કામ કરે છે, અને યુઝર ઇન્ટરફેસ આધુનિક કેવી રીતે બનાવ્યો.

2. દર્શન (Darshan) — બેકએન્ડ API અને ડેટાબેઝ આર્કિટેક્ચર

SPEAKING ROLE: DARSHAN

"Sir, I led the **Backend API & Database Architecture** using **PHP 8.2 and MySQL**.

1. Our backend is built as a pure **RESTful API** returning lightweight JSON responses. It follows Object-Oriented MVC principles with dedicated Controllers for Auth, Flights, Bookings, Airport Counter, and Admin Analytics.
2. For database persistence, we designed a normalized MySQL relational schema with 5 primary tables: users, flights, bookings, airport_bookings, and payments.
3. To ensure 100% immunity against **SQL Injection attacks (CWE-89)**, we exclusively utilized **PHP Data Objects (PDO)** with prepared statements and parameterized queries.
4. Furthermore, we built an intelligent **SQLite zero-config fallback**: if MySQL is offline or not installed, the PHP backend automatically falls back to an SQLite database file so the project never crashes."

ગુજરાતીમાં યાદ રાખો: બેકએન્ડ PHP 8.2 REST API છે, MySQL માં 5 ટેબલ્સ છે, હેકિંગથી બચવા PDO Prepared Statements વાપર્યા છે, અને SQLite ઓટો-ફોલબેક સેટ કર્યું છે.

3. ક્રિશ (Krish) — સિક્યુરિટી, બાયોમેટ્રિક ફોટો, ચેક-ઇન અને ડિપ્લોયમેન્ટ

SPEAKING ROLE: KRISH

"Sir, my focus was on **Full-Stack Integration, Security, Web Check-in, and DevOps**.

1. For authentication, we implemented stateless **JSON Web Tokens (JWT)** signed with **HMAC-SHA256**. All passenger passwords are encrypted using **Bcrypt with cost factor 10**.
2. We implemented a novel **Biometric Passenger Headshot Upload Engine**. When a passenger uploads a photo, our frontend uses HTML5 <canvas> to downscale it to 400x400 pixels at 85% JPEG quality. This compresses 5MB images into lightweight 25KB payloads, eliminating server bottlenecks while providing high-resolution headshots for tickets and manifests.
3. I built the **Self-Service Web Check-In Module**. By entering their PNR, passengers get checked in, assigned a Gate and Terminal, and receive an **Electronic Boarding Pass with a Scannable dynamic QR Code** and their photo.
4. Finally, we prepared the system for production: the frontend is deployed on **Vercel** with global edge CDN, and the backend is deployed on **Render** with Git continuous integration."

ગુજરાતીમાં યાદ રાખો: JWT અને Bcrypt સિક્યુરિટી, બ્રાઉઝરમાં ઓટો-કોમ્પ્રેસ થતો ફોટો અપલોડ, PNR થી સ્કેનેબલ QR બોર્ડિંગ પાસ જનરેશન, અને Vercel/Render ક્લાઉડ ડિપ્લોયમેન્ટ.

3. લાઈવ પ્રોજેક્ટ ડેમો બતાવવાના ૫ સ્ટેપ્સ (Live Demo Sequence)

1. હોમ પેજ અને ફ્લાઇટ સર્ચ: `http://localhost:5173` ખોલો. મુંબઈથી દિલ્હી ફ્લાઇટ્સ સર્ચ કરીને બતાવો.
2. નવું રજીસ્ટ્રેશન અને ફોટો અપલોડ: "Sign Up" માં જઈ નામ-ઇમેઇલ લખો. કેમેરા આઇકોન પર ક્લિક કરીને પેસેન્જરનો ફોટો અપલોડ કરો (લાઈવ પ્રિવ્યૂ બતાવો). રજીસ્ટર કરતાં કોન્ફેટી (ફટાકડા) અને ૨૫૦ માઇલ્સ ક્રેડિટ થશે.
3. એરબસ A320 નો ૩-૩ સીટ મેપ: કોઈ ફ્લાઇટમાં "Book Now" દબાવી સીટ મેપ બતાવો. સીટ 12A સિલેક્ટ કરી કન્ફર્મ કરો. યુનિક PNR (જેમ કે SH-8K2P) જનરેટ થશે.
4. વેબ ચેક-ઇન અને QR બોર્ડિંગ પાસ: નેવબારમાં "Web Check-In" માં જઈ પેલો PNR નાખો. પેસેન્જરનો ફોટો, ગેટ, ટર્મિનલ અને સ્કેનેબલ QR કોડ વાળો બોર્ડિંગ પાસ ઓપન થશે. "Print Pass" દબાવી પ્રિન્ટ પ્રિવ્યૂ બતાવો.
5. એરપોર્ટ કાઉન્ટર ડેસ્ક અને એડમિન પોર્ટલ: વોક-ઇન પેસેન્જરો માટે કાઉન્ટર ડેસ્ક બતાવો, અને `/admin` માં લાઈવ રેવન્યુ તથા રૂટ એનાલિટિક્સ ચાર્ટ બતાવો.

4. એકઝામિનર / સર દ્વારા પૂછાતા મુખ્ય પ્રશ્નો અને ઉત્તરો (Top Viva Q&A)

Q1. Why React instead of traditional HTML/PHP?

Answer: "Sir, React uses a Virtual DOM and Single Page Application (SPA) architecture. It eliminates full-page reloading, making flight searches, seat selection, and check-in instantaneous with high UI responsiveness."

Q2. Why PHP REST API instead of Node.js?

Answer: "Sir, PHP 8.2 has native JIT (Just-In-Time) compilation and exceptional low-memory footprint. By structuring it as an Object-Oriented REST API using PDO, we get full decoupling and high speed, which easily runs on standard Linux/cPanel and cloud containers."

Q3. How does JWT (JSON Web Token) authentication work?

Answer: "Sir, upon login, the server signs a cryptographic token containing the user ID and role using HMAC-SHA256. The client passes this token in the Authorization: Bearer header. Because it is stateless, the server doesn't maintain session memory, ensuring horizontal scalability."

Q4. How did you implement the 3-3 Airbus Cabin layout?

Answer: "Sir, we modeled 25 rows with seats A, B, C on the left and D, E, F on the right. In React, each seat maintains state: Available, Booked, or Selected. When booked, seat IDs are saved in the flight's booked_seats JSON array in MySQL, disabling them for subsequent users."

Q5. How do you prevent two passengers from booking the same seat simultaneously?

Answer: "Sir, in PHP BookingController, seat reservation executes inside a **Database Transaction** with PDO. Before inserting, it verifies seat availability. If a collision occurs, the transaction rolls back, preventing double booking."

Q6. How does the passenger photo upload work without overloading server storage?

Answer: "Sir, we use client-side HTML5 <canvas> downscaling in React. It scales the image to max 400x400 pixels at 85% JPEG quality. This reduces a 5MB image to just ~25KB while maintaining sharp facial clarity for boarding passes."

Q7. What data is stored in the Boarding Pass QR Code?

Answer: "Sir, it encodes an IATA-compliant JSON string containing PNR, Flight Number, Passenger Name, Seat Number, Gate, and 'VERIFIED_BOARDING' status, which airport scanners can read instantly."

Q8. How do you deploy this project to the cloud?

Answer: "Sir, the React frontend is deployed on **Vercel** with global Edge CDN, and the PHP backend is deployed on **Render** as a web service running `php -S 0.0.0.0:$PORT index.php` with remote MySQL database integration."

***** BEST OF LUCK TO SAHIL, DARSHAN, AND KRISH! *****